

Project Report: FinTech Fraud Detection Pipeline

Applied Big Data Engineering (Mini Project Assessment)

Malisha A.P.D. (EG/2020/4065) Lelwala L.G.S.R. (EG/2020/4047)

1 System Architecture (Lambda)

The solution is based on Lambda Architecture consisting of a real-time low latency processing (Speed Layer) and batchwise processing for analytical reports (Batch Layer).

1.1 The Data Ingestion Layer

The ingestion pipeline is the starting point for all financial events. `producer.py` - A Python based simulator to generate credit card transactions (JSON) on the fly. This script programmatically inserts anomalous events with a 10% probability of a transaction to be fraudulent. These payloads are synthetic with some key data points like user id, timestamp, merchant category, amount, location, etc. The data is written to a distributed Apache Kafka topic called transactions. Kafka is an immutable high throughput buffer that decouples fast data generation and downstream consumers.

1.2 The Speed Layer (Real-Time Stream Processing)

The Speed Layer is powered by Apache Spark Structured Streaming. Events are streamed into the Kafka topic and Spark ingests them in microbatches. The streaming job uses on-the-fly logic to parse the JSON and evaluate the transactions against the fraud rules you define.

- **Stateless Processing:** The high-value transactions are identified with simple, stateless filter (`amount > 5000`)
- **Stateful Processing:** The impossible travel detection uses a stateful stream-to-stream self-join. Spark stores the recent transaction history of users in memory and checks the incoming events against previous events for impossible geospatial movement within a 10-minute threshold.

Once identified, these fraudulent transactions are immediately sent to a PostgreSQL database sink.

1.3 The Batch Layer (Storage and Orchestration)

At the same time, the Spark streaming job stores all raw transactions into a Data Lake in the very efficient, columnar Parquet format. This satisfies the Batch Layer requirement for immutable historical records. To process this raw data, we use workflow orchestration with Apache Airflow. There is an Airflow DAG (reconciliation dag.py) scheduled to run every 6 hours. This ETL (Extract, Transform, Load) pipeline does the following:

1. **Filtering:** The raw Parquet files are read and the PostgreSQL database is queried to determine which of the transactions were already flagged as fraud by the Speed Layer.
2. **Warehousing:** The Data Warehouse receives the validated (non-fraudulent) transactions in a specific directory for clean data storage.
3. **Reporting:** It automatically creates complete analytical text and CSV reports, such as the “Reconciliation Report” (comparison of total ingress volume vs validated volume) and the “Analytic Report” (review of fraud attempts mapped against specific merchant categories).

1.4 The Serving and Visualization Layer

To combine the outputs of both Speed and Batch layers, an interactive Streamlit dashboard was created. This dashboard connects to the PostgreSQL database and displays auto refreshing, real-time metrics (total alerts, alert timelines, etc.). It also includes DuckDB to query the backend Parquet files directly, which allows analysts to do deep historical explorations of transaction volumes, merchant statistics and geospatial data without having to use a heavy external data warehouse.

2 Justification of the Tools Selected

The technology stack was selected to mirror industry standards for scalable, fault-tolerant Big Data engineering, in particular created to satisfy the high-velocity demands of the financial industry.

- **Apache Kafka for Ingestion:** Kafka was chosen as the message broker for its outstanding throughput, durability and publish-subscribe mechanics. In a FinTech environment, transaction volumes can grow unexpectedly. Kafka’s partitioning makes it easy to absorb these spikes. Crucially, the consumer group model enables a single stream of transaction data to be “fanned out”; the Spark Streaming job can generate real-time alerts and push the raw data to the data lake at the same time without affecting the performance of the message queue.
- **Apache Spark (Structured Streaming) for Processing:** Spark was chosen over other options like Apache Storm mainly because of its unified batch and streaming API and more sophisticated state management capabilities. The “Impossible Travel” rule requires complex event processing, comparing an event against another event that occurred as far back as 10 minutes ago. Spark Structured Streaming handles this nicely with stream-stream joins and built-in watermarking, managing state

in memory and clearing out state that is no longer needed, thus preventing OOM errors.

- **Apache Airflow for Orchestration:** Airflow provides resilient, Python-based DAGs to orchestrate the workflows. The batch layer requires strict dependency management, e.g. the Reconciliation Report cannot be generated in a correct way until the raw data has been successfully filtered. Airflow Schedule Model Reliable, Idempotent Execution every 6 hours.
- **PostgreSQL, Parquet, and DuckDB for Storage and Analytics:**
 - **PostgreSQL:** Chosen as the operational sink for the Speed Layer. Relational databases are great at serving highly structured, indexed data to front-end applications with low latency.
 - **Parquet:** Selected for the Data Lake due to its columnar nature limiting disk IO. Parquet’s orders of magnitude better performance is due to analytical queries reading only a handful of columns as opposed to row-based formats such as CSV or JSON.
 - **DuckDB:** An in-process analytical SQL engine, embedded directly into the Streamlit app. The intention is that it can query Parquet files directly, without loading huge data sets into Pandas DataFrames.

3 Explanation of Event Time vs. Processing Time

In the world of distributed stream processing, not distinguishing between Event Time and Processing Time is a critical architectural mistake, especially for fraud detection.

- **Processing Time** is the wall-clock time from when the Spark cluster receives the data packet and processes it.
- **Event Time** is when the credit card was actually swiped in the physical or digital world and is represented by the timestamp created by the point-of-sale system and embedded within the JSON payload.

3.1 The Danger of Processing Time in Fraud Detection

Processing Time is susceptible to unforeseen network latency resulting in drastic logical failure. Suppose a user has a valid transaction in London at 12:00 PM, and another in New York at 6:00 PM. For example, if the London transaction has a huge network delay and arrives at the Kafka broker at 6:01 PM, a Processing Time based system would see two transactions arriving within one minute from two different continents. This would incorrectly flag this as “Impossible Travel” resulting in a frozen account and a very unhappy customer.

3.2 Handling Event Time via Watermarking

The pipeline was designed with strict Event Time semantics to ensure determinism and correctness. First, the timestamp field from the Kafka JSON payload is explicitly parsed

and cast to a Spark TimestampType (event time). All windowing and joining activities are based on this column. Secondly, we implemented a Watermark of 10 minutes (.withWatermark("event time", "10 minutes")) Watermarking is an approach to define the lateness of a transaction that can be handled by Spark for stateful aggregation.

- **Late Data Handling:** If the latest global maximum event time seen by the system is 12:30 PM, the 10-minute watermark tells Spark to accept and process out-of-order events with timestamps as late as 12:20 PM.
- **State Expiration:** At the same time, the watermark also triggers memory cleanup. When the watermark is 10 minutes ahead of an event's timestamp, Spark can safely remove the location state of that user from the memory of the cluster.

Using Event Time and watermarks, the stream-to-stream self-join measures the real chronological distance between transactions, so the "Impossible Travel" alert is only triggered by real physical impossibilities, not network lag.

4 Ethics Section: Privacy Implications & Data Governance

4.1 Privacy Implications of User Profiling

Real-time fraud detection offers a crucial security for consumers and financial institutions, but at the price of a massive apparatus of pervasive tracking and behavioral profiling. The data pipeline in FinTech or E-commerce ecosystem deals with very sensitive attributes like accurate geospatial coordinates (location, country), temporal habits (timestamp), financial capability (amount), behavioural preferences (merchant category).

The continuous aggregation of this data creates a highly detailed, intrusive map of a user's life.

- **Surveillance and Movement:** Institutions know where users make every purchase, so they have a real-time GPS tracker on their users, they know their daily commutes, their international travel, and their habitual patterns.
- **Behavioral Inference:** Algorithms can infer very intimate behaviors by analyzing spending through merchant categories (healthcare, entertainment, luxury goods, etc.). A sudden increase in healthcare and pharmacy transactions might indicate a medical condition before the user has even told anyone.
- **Re-identification Risks:** The datasets do not contain explicit identifiers such as names, but the unique combination of geolocation, timestamps, and specific purchase amounts creates a digital fingerprint that can easily be cross-referenced to re-identify individuals.

4.2 Data Governance and Mitigation Strategies

To tackle these serious ethical and privacy challenges, strict data governance frameworks need to be built directly into the architecture of the data pipeline.

1. **Data Minimization and Pseudonymization:** The pipeline has to adhere to the principle of data minimization. The system shall not ingest or store personally identifiable information (PII) in plaintext. For example, the user id should be hashed (irreversibly) (e.g., SHA-256 with a unique salt) at the edge.
2. **Strict Retention Policies (Data Lifecycle Management):** The architecture currently stores raw events forever in the Parquet files. Good data governance says you should automate Time-to-Live (TTL) policies. This is the time to automatically purge or move the highly granular raw data to heavily encrypted cold storage once the Airflow batch layer has successfully reconciled the data.
3. **Role-Based Access Control (RBAC):** Access to different layers of the Lambda architecture needs to be tightly controlled. Aggregated anonymized Airflow reports should only be accessible by business analysts and marketing teams.
4. **Regulatory Compliance and User Autonomy:** The infrastructure must operate in strict compliance with international data protection laws, such as the GDPR or CCPA. This means the system is built to enable user autonomy, empowering customers to exercise their "Right to be Forgotten" to delete their historical transaction data when closing their accounts.

5 Conclusion

This project shows the power and necessity of the Lambda Architecture in modern day data engineering. This pipeline uses Kafka, Spark Streaming, Airflow and a hybrid storage layer (PostgreSQL and Parquet) to achieve the microsecond response needed to intercept financial fraud and the batch reliability required for historical auditing. In addition, the system is designed to be robust to the realities of networks by acknowledging and designing around the complexities of Event Time.